Name: Shahzeb Mehdi
FDU-ID: 2124332

Subject: Programming for the Internet
Mid-Term Project

E-Commerce Microservices

Introduction

This project implements a **containerized distributed backend system for a simplified e-commerce platform** using ASP.NET Core microservices. In this project I have implemented a **microservices-based architecture with database-per-service**, containerize the services using Docker, and enable communication between services to perform core e-commerce operations such as managing products, customers, orders, and payments.

System Architecture Overview

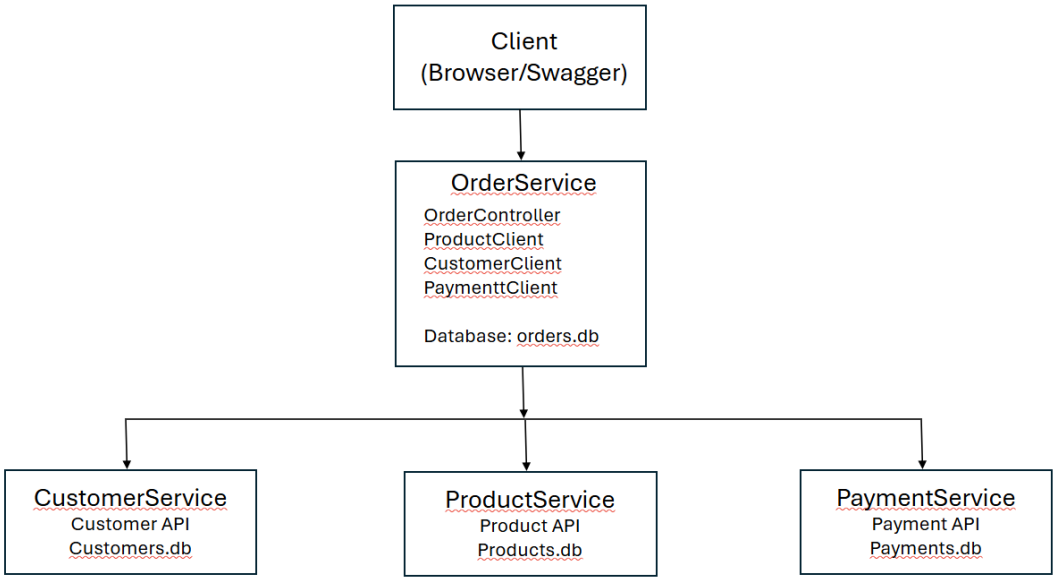
The application is composed of **four independent microservices**, each responsible for a specific domain.

Implemented Microservices

1. **Product Service**
    a. Manages product catalog
    b. Maintains product price and stock
    c. Updates stock when an order is placed
2. **Customer Service**
    a. Manages customer information
    b. Stores customer details
3. **Order Service**
    a. Creates orders
    b. Validates customer and product existence
    c. Calculates order totals
    d. Coordinates communication between services
4. **Payment Service**
    a. Handles order payment processing
    b. Records payment transactions

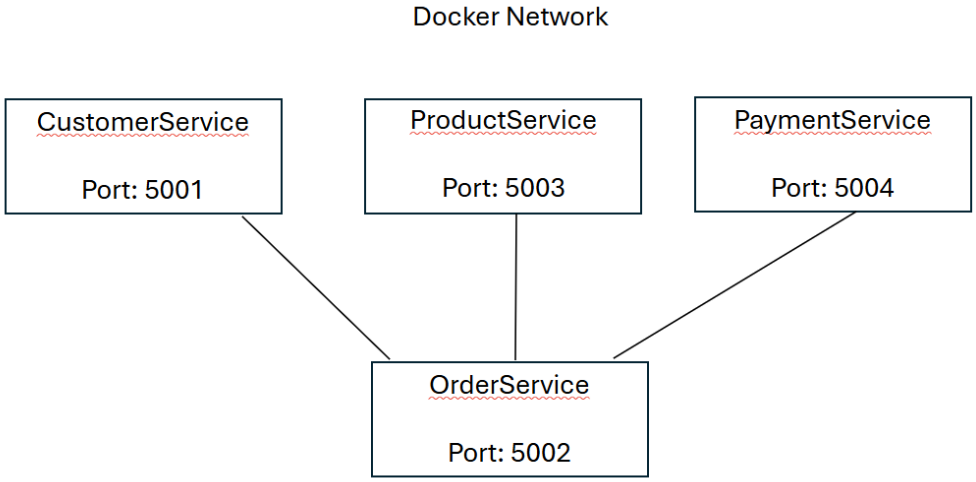
Each service runs independently in its own container and maintains its own database.

Architecture Diagram



Docker Container Deployment

docker-compose.yml



All services run inside **Docker containers** and communicate using Docker's internal network

The **Order Service acts as the orchestration layer**, coordinating interactions between other services.

Database per Service Implementation

The system follows the **Database-per-Service pattern**, where each microservice owns its own database and schema.

Service	Database
CustomerService	customers.db
ProductService	products.db
OrderService	orders.db
PaymentService	payments.db

Each service contains its own:

- **DbContext**
- **EF Core migrations**
- **SQLite database**

This approach ensures **data isolation and prevents tight coupling between services**.

Example configuration in ProductService:

```
builder.Services.AddDbContext<ProductDbContext>(options =>
  //options.UseSqlite("Data Source=products.db"));
  options.UseSqlite("Data Source=Data/products.db"));
```

EF Core Asynchronous Data Access

All services use **asynchronous EF Core methods** to ensure scalable and non-blocking data access.

Examples used throughout the project:

```
await _db.Products.ToListAsync();
await _db.Products.FirstOrDefaultAsync();
await _db.Products.AnyAsync();
await _db.SaveChangesAsync();
```

Using asynchronous methods improves system responsiveness and allows better resource utilization.

Service Communication (Synchronous)

The system implements **synchronous communication using HttpClient**, where the Order Service communicates with other services to validate and process orders.

OrderService performs the following steps when creating an order:

1. Validate that the **customer exists** via CustomerService.
2. Validate that the **product exists** via ProductService.
3. Retrieve product details and calculate order total.
4. Create the order in the Order database.
5. Call PaymentService to process payment.
6. Update product stock via ProductService.

Example ProductService call:

```
var product = await _productClient.GetProductAsync(req.ProductId);
```

Example CustomerService validation:

```
var exists = await _customerClient.CustomerExistsAsync(req.CustomerId);
```

Containerization with Docker

Each microservice is containerized using a **multi-stage Dockerfile**.

Example Dockerfile structure:

Stage 1: Build

- Restore dependencies
- Build application

Stage 2: Runtime

- Copy published output
- Run ASP.NET Core application

Example runtime command:

```
ENTRYPOINT ["dotnet", "OrderService.Api.dll"]
```

This approach ensures lightweight production-ready containers.

Docker Compose Orchestration

All services are started together using **Docker Compose**, which defines service containers, networking, and volumes.

Example command to start the system:

```
docker compose up --build
```

Services defined in docker-compose.yml:

- customerservice
- productservice
- orderservice
- paymentservice

Each service exposes an API port:

Service	Port
CustomerService	5001
OrderService	5002
ProductService	5003
PaymentService	5004

Docker networking allows services to communicate using container names such as:

```
http://productservice:8080
```

Data Ownership and Coupling Prevention

The system ensures **clear data ownership**, where each microservice manages its own data.

Examples:

- ProductService manages product catalog and stock.
- CustomerService manages customer data.
- OrderService manages order records.
- PaymentService manages payment transactions.

Other services **cannot directly access another service's database**. Instead, they interact through APIs.

This approach prevents tight coupling and maintains service independence.

Code Organization

The project follows a clean and modular structure.

```
CODEBASE
│
├ CustomerService
│   └ CustomerService.Api
│
├ ProductService
│   └ ProductService.Api
│
├ OrderService
│   └ OrderService.Api
│
├ PaymentService
│   └ PaymentService.Api
│
└ docker-compose.yml
```

Each microservice contains:

- Controllers
- Models
- Services
- DbContext
- Migrations
- Dockerfile

This ensures maintainable and scalable code organization.

Order Processing Workflow

Example order creation workflow:

1. Client sends request to **OrderService**.
2. OrderService verifies customer via **CustomerService**.
3. OrderService verifies product via **ProductService**.
4. Product information is retrieved.
5. OrderService calculates order total.
6. OrderService creates order record.
7. PaymentService processes payment.
8. ProductService updates product stock.

This demonstrates a **coordinated microservices workflow**.

Conclusion

This project successfully demonstrates how to build a **containerized microservices backend for an e-commerce platform** using ASP.NET Core and Docker.

Key architectural principles implemented include:

- Microservices architecture
- Database-per-service pattern
- Service-to-service communication
- Containerized deployment using Docker
- Data ownership and loose coupling

The system illustrates how distributed backend systems can be designed to support scalability, modularity, and maintainability.